Computers & Security 154 (2025) 104420

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

LowPTor: A lightweight method for detecting extremely low-proportion darknet traffic☆

Qiang Zhang, Cheng Huang *, Jiaxuan Han, Shuyi Jiang, Jiayong Liu

School of Cyber Science and Engineering, Sichuan University, Chengdu, 610065, Sichuan, China

ARTICLE INFO

Keywords:
Darknet
Tor
Traffic detection
Lightweight neural network
Operator fusion

ABSTRACT

The darknet's accessibility has increased significantly in recent years, making it more susceptible to exploitation by cybercriminals. The darknet's inherent characteristics of high concealment, strong anonymity, and resistance to tracing have created a fertile ground for illicit activities, which are becoming increasingly prevalent. Deep learning models' complex computations in high-dimensional feature spaces result in significant computational overhead and prolonged inference times, hindering their deployment in backbone networks where real-time detection is crucial. Dealing these challenges requires innovative solutions for efficient traffic sampling, feature engineering, model simplification, memory optimization and inference acceleration. Therefore, this paper introduces a collaborative filtering algorithm, an extensible channel-wise feature group, and a lightweight detection algorithm, designed to optimize performance through operator fusion and channel alignment. The proposed algorithm combines the advantages of lightweight CNNs (Convolutional Neural Networks), leverages GPU (Graphics Processing Unit) parallelism, and reduces memory allocation overhead, resulting in a CUDA (Compute Unified Device Architecture) core-optimized neural network model that achieves significant inference speedup. We utilize extensible channel-wise feature group derived from short traffic packet sequences to improve detection accuracy. Our approach targets the detection and prevention of illicit darknet traffic during the connection phase, rather than interfering with data transmission. By integrating rule-based filtering with small flow sampling within collaborative filtering, we facilitate early detection while maintaining minimal complexity overhead. To the best of our knowledge, the methodology proposed in this paper is the first to be designed based on operator fusion and channel alignment strategies, specifically aimed at detecting extremely low-proportion darknet traffic within backbone networks. Our approach synthesizes three extremely low-proportion darknet traffic datasets, utilizing the self-built, CIC-Darknet2020, and TCUB datasets as Tor sources. Notably, our approach achieves a 45.79% reduction in actual inference time compared to current state-of-the-art (SOTA) method, while maintaining SOTA detection accuracy. Furthermore, our method exhibits the capability to filter out up to 91.26% (with a minimum of 78.01%) of the packets to be processed, without compromising any flows.

1. Introduction

In recent years, with the continuous expansion of network scale, Internet Service Providers (ISPs) need to conduct network monitoring and detect network security threats in complex network environments. This monitoring and protection in backbone networks are crucial for ensuring the security and stability of the Internet. The darknet, primarily supported by Tor (The Onion Router) technology (Dingledine et al., 2004), harbors many criminal activities. Therefore, monitoring darknet traffic in backbone networks has become a key focus in detecting darknet criminal behavior. However, traditional Deep Packet Inspection (DPI) technology (Bujlow et al., 2015) is inefficient in handling large-scale traffic, making real-time processing difficult. Additionally, traffic engineering must address the challenges of the extremely low proportion of darknet traffic and the sudden bursts of other types of traffic, further complicating darknet traffic detection in backbone networks. In actual ISP deployments, using DPI methods to process large amounts of traffic is impractical. Therefore, flow-based monitoring methods are typically used, representing all communications in the form of flows. In the realm of computer networks, a "flow" is

☆ This document is the results of the research project funded by the National Key Research and Development Program of China (No. 2023YFB3106600) and Sichuan Province Science and Technology Support Program, China (No. 2024ZHCG0175).

* Corresponding author.
E-mail address: codesec@scu.edu.cn (C. Huang).

https://doi.org/10.1016/j.cose.2025.104420
Received 2 January 2025; Received in revised form 27 February 2025; Accepted 5 March 2025
Available online 13 March 2025
0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

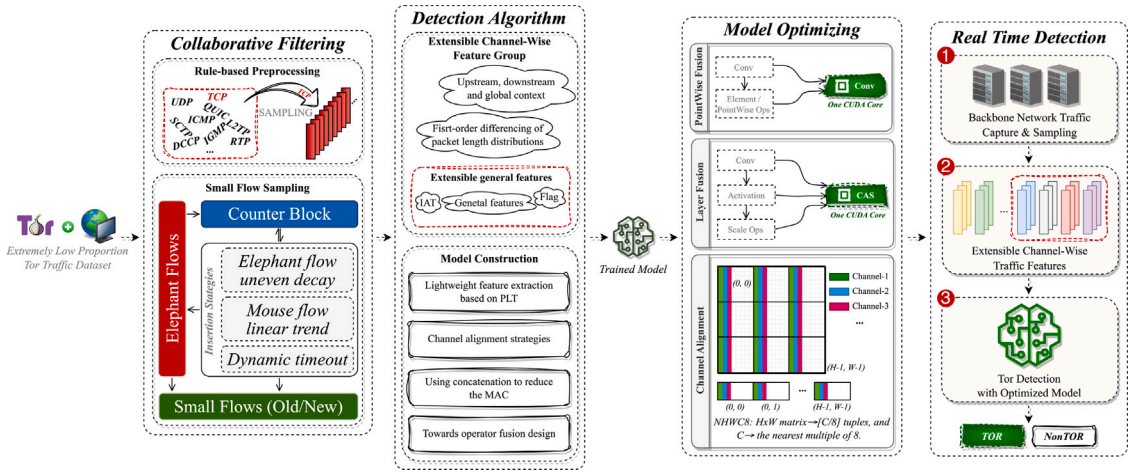

Fig. 1. Flowchart of the real-time lightweight detection method for extremely low-proportion darknet traffic in backbone networks. PLT means Partial linear Transformation. NHWC8 refers that the values of all channels in the HxW matrix are packaged into a tuple of [C/8], with C rounded to the nearest multiple of 8.

defined as a continuous sequence of packets that share an identical five-tuple—comprising the source IP (Internet Protocol) address, source port, destination IP address, and destination port. This aggregation of packets, both upstream and downstream, is treated as a singular entity for the purposes of network management and analytical processes. Many detection schemes have employed extended flows combined with various techniques, including Machine Learning (ML) and Deep Learning (DL). However, these methods mainly focus on model accuracy while neglecting practical usability. Consequently, although existing research has made progress in detection accuracy, the feasibility and effectiveness in practical applications remain pressing issues.

Previous research has mainly relied on experiments with small-scale, time-sequenced collected traffic datasets. This approach overlooks the complexity of extremely low-proportion darknet Tor traffic mixed with other large-scale traffic in actual networks, rendering these methods less applicable in real network environments (Wu et al., 2021). To effectively handle the challenges posed by large-scale traffic, sampling operations are typically performed to reduce the data volume that needs processing. However, even after sampling, a substantial amount of traffic data remains. Therefore, the dimensions of traffic features, model size, and inference speed must all meet strict real-time requirements to ensure the method's effectiveness and deployability. In terms of feature extraction for encrypted darknet Tor traffic, traditional methods are insufficient, necessitating more complex techniques, including modeling the behavior of encrypted algorithm communications, analyzing the statistical characteristics of encrypted data, exploring the transformation patterns of encryption algorithms, and the behavioral patterns of different protocols (Al-Omari et al., 2022). These changes make most traditional traffic detection and classification methods less effective when dealing with encrypted darknet traffic. Currently, commonly used methods for detecting and classifying encrypted darknet Tor traffic are mainly divided into ML-based methods and DL-based methods. ML-based detection methods rely on manually extracted features, which require domain experts to invest significant time and effort. However, with the development of encryption technology, traffic features have become increasingly difficult to capture and analyze, making feature selection more challenging. If feature selection is inappropriate, it may lead to poor model performance. In contrast, DL-based detection methods excel in identifying complex patterns and features, but they demand high computational resources, have poor interpretability, and rely heavily on large-scale labeled network traffic data. Thus, although DL methods have advantages in detecting complex patterns, their high computational costs and deployment difficulties pose many challenges for practical deployment.

In general, the payload values in encrypted traffic packets become highly random after encryption, making port-based and payload-based methods very difficult to analyze data effectively (Velan et al., 2015). In recent years, researchers have shown increasing interest in end-to-end DL methods, which use raw traffic data as input and fully automate the feature extraction process. Despite progress in existing research, current SOTA DL-based methods still face significant challenges, such as high feature dimensionality, poor real-time performance, and poor deployability. The reason why it is difficult to use common neural network structures such as CNNs and Transformers (Vaswani et al., 2017) for darknet Tor traffic detection is that, although CNNs have excellent capability in extracting local features and Transformers can construct long-distance, global dependencies, their enormous parameter sizes limit the feasibility of deploying them in real ISP environments. For most models that perform well in offline environments but are not deployable in production environments, the slow inference speed problem is not only due to the huge performance requirements caused by the unreasonable model design but also because some models, despite reducing parameter counts and FLOPs (Floating Point Operations), neglect MAC (Memory Access Cost) and parallelism, resulting in significant differences between FLOPs and actual inference speeds.

In this paper, we propose a collaborative filtering algorithm, extensible channel-wise feature group, and a network structure and sequence optimized based on operator fusion and channel alignment strategies, rather than just using TensorRT[1] technology to optimize the operators after model training. This design, based on operator fusion and channel alignment, can fully utilize the parallel computing power of the GPU, reduce repeated memory allocation, and significantly enhance inference speed. The overall architecture of our proposed method is shown in Fig. 1. Compared to existing research, the contributions of this paper can be summarized as follows:

- A collaborative filtering algorithm has been proposed to tackle the challenge of extremely low-proportion of darknet traffic in backbone networks.
- Proposed extensible channel-wise feature group that encompass upstream, downstream, and global context. This design effectively encapsulates the behavioral characteristics of darknet Tor traffic while keeping the model input within a lightweight range, and it also provides the capability to extend to other encrypted traffic detection applications.
- Design a lightweight detection algorithm based on the maximization of operator fusion and channel alignment, thereby not only reducing the number of parameters and FLOPs but also decreases MAC, significantly improving the model's actual inference speed.

[1] https://docs.nvidia.com/deeplearning/tensorrt/archives/tensorrt-1020/developer-guide/index.html#enable-fusion.

- Create a darknet traffic dataset characterized by an extremely low-proportion of Tor traffic, mirroring the real-world scarcity of Tor traffic in backbone networks.

The rest of this paper is organized as follows: The second part mainly explains related work on network traffic sampling, darknet and encrypted traffic detection, lightweight neural network design. The third part elaborates on the overall architecture of the proposed method, collaborative filtering, extensible channel-wise feature group, and the detailed design and implementation of lightweight neural networks based on operator fusion and channel alignment strategies. The fourth part introduces the datasets used and the experimental configuration. The fifth section undertakes a comprehensive comparative analysis of the relative merits of collaborative filtering and extensible channel-wise feature group, complemented by a thorough examination of the detection model's performance via ablation studies and an assessment of inference speed. Ultimately, the sixth section distills the key findings of the paper and offers a forward-looking perspective on promising directions for future investigation.

2. Related work

2.1. Traffic sampling

In backbone networks, the bandwidth is typically substantial, posing challenges for traffic analysis software to process all traffic data in real-time. Therefore, it is often necessary to first utilize DPDK[2] and IPFIX (IP Flow Information Export)[3] to capture high-speed traffic before conducting traffic sampling. Existing sampling techniques can be classified into packet-based and flow-based categories (Wu et al., 2021). In real-world network environments, the volume of traffic data is immense, with a notable presence of both small flows (Mouse Flows) and large flows (Elephant Flows) (Yang et al., 2018). Small flows are characterized by a high number of flows containing a limited number of packets, whereas large flows consist of fewer flows with a substantial number of packets. The proportion of Tor traffic within the network is particularly minimal and predominantly comprises small flows. However, with prolonged use of Tor, the frequency of packets sharing the same quintuple (source IP, destination IP, source port, destination port, and protocol type) increases (Valenti et al., 2013), leading to the potential transformation of some small flows into large flows. Consequently, after a certain duration of the sampling period, the packets collected primarily originate from large flows. In backbone networks, effective traffic sampling techniques can significantly reduce the volume of data processed while still capturing essential traffic information, thereby facilitating efficient network management and subsequent traffic analysis.

2.2. Encrypted traffic detection

Currently, the mainstream darknet include the Tor network (Dingledine et al., 2004), the I2P network (Abdo and Hossain, 2023), the ZeroNet network (Hu et al., 2020), and the FreeNet network (Hu et al., 2020; Clarke et al., 2001). Among these common anonymous communication networks, the Tor network is the most widely used. The technologies utilized in the implementation principles of darknet include onion routing (Goldschlag et al., 1999), garlic routing (Abdo and Hossain, 2023; Maalavika et al., 2023), protocol obfuscation and traffic obfuscation techniques (such as Obfs4, Meek protocol, etc.) (Angel and Winter, 2014), data encryption, anonymous authentication, distributed networking, and P2P technology. Onion routing involves wrapping the content a user wants to send into a multi-layer encrypted data packet, similar to an onion, and then transmitting it through a series of network nodes called onion routers. Each onion router decrypts one layer of encryption until the destination is reached. After the final layer is decrypted, the destination user can access the original data. Garlic routing is a variant of onion routing, where the original data to be sent is split into multiple encrypted data packets and transmitted through multiple intersecting tunnels. This method makes traffic detection and analysis difficult for attackers or regulators because, in garlic routing, the encrypted data streams for uploading and downloading typically use different tunnels, and each direction may have multiple different tunnels.

For the traffic generated during the connection establishment and data transmission processes of Tor, various methods have been used to process raw traffic and obtain its characteristics. These methods include sparse matrices, raw bytes, the number of packets in the upstream and downstream of a single flow, time characteristics, and other statistical features. LEXNet (Fauvel et al., 2023) formalizes encrypted traffic classification as a multivariate time series (MTS) classification task. Its input consists of packet-level data such as packet size and direction related to the first 20 packets of a single flow, and its output is an application label. MTC (Wang et al., 2023), to reduce the burden of data preprocessing and save computational and storage resources, selects a single packet as the input to the model. This method removes physical layer information, features with minimal differences, and redundant information, cleans anomalies such as NaN values, and normalizes the data. Finally, it pads packets of different lengths to a fixed size of 1500 bytes, the default value of the Maximum Transmission Unit (MTU) for Ethernet frames, as the model input. NetTiSA (Koumar et al., 2024) sets an activity timeout of 300 s and an inactivity timeout of 65 s during the traffic extraction process from PCAP (Packet Capture) files. It uses 0-padding for certain features that cannot be computed due to insufficient data in short single-packet flows. They compute classification features in three groups: the first group is based on classical bidirectional flow information, such as transmitted bytes and packet counts; the second group includes statistical and time-series analysis-based features calculated by analyzing packet sequences; the third group features are derived from the first two groups, enhancing classification performance without significantly increasing performance overhead. NetMamba (Wang et al., 2024) divides different packets into flows, selects the first M packets in each flow, removes irrelevant information from each packet, and standardizes the packet header and payload length to achieve a fixed-length input. Finally, it uses methods similar to time-series analysis, slicing the array of all packets in the original single flow into small segments to synchronize the basic timing information of network traffic, preserve potential relationships between bytes, eliminate bias, and reduce memory consumption.

2.3. Lightweight detection model

Various strategies have been proposed to achieve compact deep neural networks. These strategies include network pruning (Cheng et al., 2024), low-bit quantization (Wang et al., 2021), and knowledge distillation (Tang et al., 2020). Network pruning involves trimming redundant parameters in the neural network to obtain a more efficient model. Low-bit quantization converts weights and activation values from high precision (typically 32-bit floating-point) to low precision (such as 8-bit, 4-bit, or even lower integer representations) to reduce the model's storage requirements and computational complexity while maintaining performance as much as possible. Knowledge distillation transfers the knowledge from a larger model to a smaller one, helping the smaller model achieve better performance. However, the performance of these methods is often limited by the performance of the pre-trained neural network used as the benchmark model.

GhostNet (Han et al., 2020) introduces the Ghost module to reduce computation by replacing traditional convolutions with features derived from linear transformations of existing feature maps. ShuffleNet (Zhang et al., 2018; Ma et al., 2018) introduces several essential

[2] https://doc.dpdk.org/guides/linux_gsg/.
[3] https://datatracker.ietf.org/doc/html/rfc7011.

design principles for lightweight neural networks aimed at optimizing model architecture. It optimizes lightweight neural networks using point-wise grouped convolutions and a channel shuffle mechanism for better feature exchange. CondenseNet (Huang et al., 2018; Yang et al., 2021) enhances inter-layer connections with dynamic weight pruning, while CondenseNetV2 (Yang et al., 2021) introduces a learnable sparse feature reactivation module and employs matrix decomposition and SE (Squeeze-and-Excitation) lightweight attention to optimize feature selection and enhancement. EfficientNet (Tan and Le, 2019) points out that although depth-wise separable convolutions theoretically have fewer parameters and lower FLOPs, their computation speed is slower when used in the shallower layers of the network. The MobileNet series (Howard et al., 2017; Sandler et al., 2018; Howard et al., 2019) ensures model accuracy and lightweight efficiency through various optimizations: depth-wise separable convolutions in V1, the linear bottleneck inverted residual structure in V2 (where ResNet's residual structure (He et al., 2016) reduces dimensions first and then increases them, while MobileNetV2 increases dimensions first and then reduces them), SE lightweight attention mechanism, and the use of the NetAdapt algorithm to obtain the optimal number of convolution kernels and channels.

It is important to note that when designing network structures, especially for real-time inference performance, model parameters and FLOPs are not the sole determinants of a model's inferencing speed. Common optimization strategies currently include selecting classical neural network structures with high computational and memory access efficiency, using TensorRT custom operators, or performing operator fusion on trained models. For example, arranging consecutive compute-intensive operators in the order of $Conv - BN - ReLU$ will fuse them into a single operator, reducing the model's computational load and memory access time, but an arrangement like $Conv - ReLU - BN$ cannot directly fuse the batch normalization (BN) layer into the convolutional (Conv) layer. Additionally, convolution kernel sizes should preferably use common sizes like 5×5, 3×3, and 1×1 (Gu et al., 2018); the MAC is minimized when the input and output channels of convolution layers are equal because even if the model parameter count is the same, the inference time of the model will vary if the MAC is different (Ma et al., 2018; Yang et al., 2017); GPUs excel at parallel computing, and the larger the tensor, the higher the GPU utilization efficiency. Splitting large convolution operations into small fragmented operations will be detrimental to GPU computation. The above strategies not only reduce the model's parameter count, FLOPs, and MAC but also fully leverage the CUDA cores, thereby improving the model's inference performance.

3. Methodology

In this section, we will give a detailed description of the process of Tor traffic detection method proposed in this paper. As shown in Fig. 1, the process begins with high-speed traffic capture using DPDK, and then perform rule-based filtering by configuring the filter protocol rules using the flow rules API (Application Programming Interface) provided by DPDK. This step filters out a large portion of irrelevant traffic by virtue of the fact that Tor traffic is using the TCP (Transmission Control Protocol), as well as other protocol types that are certainly not Tor traffic. Next, the captured traffic undergoes small flow sampling process, and the sampled traffic is used to extract features for subsequent Tor traffic detection. The darknet traffic detection module primarily comprises the lightweight CNN model, designed based on operator fusion and channel alignment strategies. Before conducting inference with the final model, two critical steps must be undertaken: model training and optimization. Initially, the existing dataset undergoes feature extraction to delineate traffic features, followed by partitioning into training, validation, and test sets to facilitate the training of the target detection model. Subsequently, the trained model is exported in ONNX (Open Neural Network Exchange) format. This model is then optimized using TensorRT, which provides insights into the fusion of various network layers. Analyzing these fusion results allows for the reorganization and optimization of the structural configuration and sequencing of the network layers, iterating this process until the desired performance outcomes are achieved. In the following sections, we will introduce the overall architecture of the proposed collaborative filtering algorithm, the lightweight design of detection algorithm's each layer, operator fusion and channel alignment design in detail. These design implementations are crucial for reducing the overall numbers of parameters, FLOPs, and MAC of the model, maximizing the utilization of CUDA cores, and significantly improving the model's inference speed and detection accuracy.

3.1. Collaborative filtering

Due to the extremely low-proportion of Tor traffic in backbone networks, we perform collaborative filtering on the complete traffic that has been filtered based on rule-based preprocessing to reduce the number of packets that need to be processed. The first step in collaborative filtering is to remove traffic that does not conform to the communication patterns used by Tor traffic from the complete raw traffic. We first capture high-speed traffic in real-time using DPDK and configure rule-based preprocessing via the DPDK flow rule API to filter out common non-Tor traffic protocol types, and select packets that conform to TCP traffic. This step can significantly reduce the amount of ordinary traffic, greatly lowering the spatial and computational requirements of our proposed method. And we further increase the proportion of Tor traffic using small flow sampling, meeting the feasibility requirements for detecting extremely low-proportion Tor traffic.

3.1.1. Rule-based preprocessing

In the rule-based preprocessing phase of collaborative filtering, our primary objective is to identify and eliminate traffic that does not conform to the characteristics of the Tor network. The essence of this process lies in the effective classification and screening of traffic to ensure the accuracy and efficiency of subsequent analyses. Specifically, we employ DPDK to facilitate efficient network traffic capture and utilize its flow rule API to configure specific filtering rules. This series of rules is specifically designed to target common traffic protocols, such as QUIC, ICMP, IGMP, SCTP, DCCP, RTP, FTP, POP3, SMTP, SSH, and UDP.[4] These protocols typically dominate traditional network communications but do not align with the anonymity and privacy protection features inherent to Tor. By filtering out these protocols, we can concentrate on the TCP, which is primarily relied upon by the Tor network. The advantages of this screening process are substantial, as it significantly reduces the volume of packets that needs to be processed, thereby lowering the demands on storage and computational resources. Additionally, the reduction of ordinary traffic interference contributes to improved accuracy in subsequent analytical algorithms.

3.1.2. Small flow sampling

For practical network traffic detection requirements, the precise measurement of mouse flows, holds limited significance. However, these flows consume a substantial portion of the scarce on-chip storage capacity. Moreover, due to the inherent challenges in accurately assessing the magnitude of a flow upon the arrival of packets in real-time, it becomes imperative to pre-allocate adequate counting space for each flow. This necessity to accommodate the storage requirements of mouse flows leads to considerable resource inefficiency. In real network traffic, the sizes of flows exhibits a pronounced skew in their distribution (Li et al., 2018), indicating that the majority of flows are comparatively diminutive, whereas a select few are substantially larger. This phenomenon is frequently characterized as a "long-tail distribution", wherein a limited number of flows constitute a significant

[4] https://en.wikipedia.org/wiki/OSI_model.

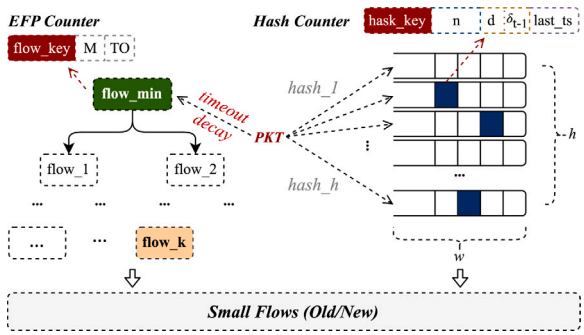

Fig. 2. The small flow sampling algorithm consists of a elephant flow priority (EFP) counter and a hash counter, where k represents the number of elephant flows in the queue, M represents the number of packets in the EFP counter of the current flow, n represents the number of packets in the hash counter, TO indicates whether the flow has timed out, d indicates the duration of the flow, δ_{t-1} corresponds to the previous timeout threshold, and w and h denote h arrays of length w.

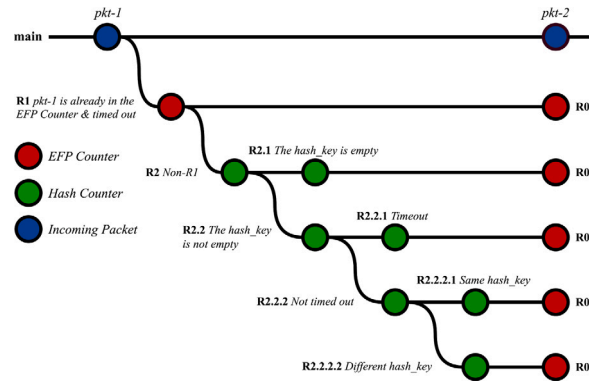

Fig. 3. The diagram of small flow sampling process, where R is the abbreviation for rule, and the horizontal axis represents the logical branches of processing.

proportion of the bandwidth or data volume, while the predominant majority of flows remain relatively modest in size. Packet sampling is extensively utilized to generate flow records in backbone networks. However, random sampling, which selects 1 in N packets (systematic sampling) (Wu et al., 2021), is prone to omitting a significant number of flows, particularly the predominant short flows (those consisting of fewer than N packets). Consequently, we introduce a sampling methodology specifically designed for small flows, which significantly increases the probability of capturing them, while generally discarding elephant flows.

In backbone networks, accurately tracking all packets of network flows is nearly impossible. Consequently, the objective of our proposed small flow sampling methodology is to eliminate elephant flows while differentiating between old flows and newly initiated flows sharing the same five-tuple (both upstream and downstream). Small flows, which are temporarily classified as elephant flows, will be attenuated to make room for genuine elephant flows. A dynamic timeout strategy is employed to expedite this process, and allowing for the differentiation between new and old flows of the same flow. The specific process is illustrated in Fig. 2. In the hash counter, the attenuation of the flow's n and δ in the h arrays is dynamically accelerated. The n attenuation for small flows occurs rapidly, while that for elephant flows is gradual. Conversely, the δ attenuation for small flows is slow, whereas for elephant flows, it is swift. The design of the acceleration for n attenuation significantly impacts small flows while exerting minimal influence on elephant flows; the opposite is true for δ attenuation. Through this strategy, the n value corresponding to small flows can be easily reduced to zero and replaced by other flows, while the n value for elephant flows can be readily increased to a substantial value, allowing them to be recorded in the elephant flow priority counter.

Moreover, as time progresses, the timeout threshold diminishes, further accelerating the statistics of elephant flows. This approach is beneficial because, due to the skewed nature of traffic distribution in backbone networks, tracking elephant flows occupies significantly less space compared to monitoring small flows. Additionally, once a flow has been identified as elephant flow, there is no need for continued tracking. Timed-out flows will be reset in the hash counter. By utilizing hash keys instead of bit arrays like those in bloom filters, we can effectively perform implicit clearing of counters without the concern of existing flows accumulating in the counters over time, which could lead to saturation and an increased false positive rate. Consequently, the adoption of a dynamic timeout strategy, which differentiates between new and old flows sharing the same five-tuple, can significantly enhance the efficiency of elephant flow tracking while concurrently minimizing resource overhead. The process of the small flow sampling method is illustrated in Fig. 3, and its core strategies can be divided into the following scenarios:

R1 Existence of Timed Out Flows in the EFP Counter: If there are any timed out flows, skip to the next packet; if not, proceed to the hash counter process.

R2.1 Presence of Corresponding Hash Key in the Hash Counter: If the corresponding hash key is absent, initialize the relevant parameters; if present, implement the decay of δ and n according to the established process.

R2.2.2 Decay of the δ Under Non-Timed-Out Conditions: With a probability of $1-(p_1)^d$ $(p_1 < 1)$, decrement the current δ by 1, as shown in Algorithm 1. The dynamic timeout threshold δ comprises two components: the δ_{t-1} from the previous moment of the current flow and the damped global linear trend value $(l_t + b_t)$, which reflects the time difference between the current packet and the previous packet in the global context, as described in Eq. (1),

$$\hat{\delta}_t = \hat{\delta}_{t-1} + l_t + b_t, \tag{1}$$

where l_t is an estimate of the level of the packet time series at time t which satisfies

$$l_t = \alpha\tau_t + (1-\alpha)(l_{t-1} + \alpha b_{t-1}), \tag{2}$$

and b_t an estimate of the trend of the packet time series at time t which satisfies the trend update formula

$$b_t = \beta(l_t - l_{t-1}) + (1-\beta)\beta b_{t-1}, \tag{3}$$

both α in Eq. (2) and β in Eq. (3) are smoothing parameters and $\beta = \alpha$ $(\alpha, \beta \in [0,1])$. The expression τ_t denotes the time difference between the current incoming packet and the previous packet in the same flow.

R2.2.2.2 Decay of n Under Inconsistent Hash Keys: With a probability of ${p_2}^n$ $(p_2 < 1)$, decrement n by 1. The details regarding the variation of the n can be found in Algorithm 1.

R0 Update Strategy for the EFP Counter: If a value already exists in the EFP counter, update it directly to the maximum value stored in the corresponding hash key of the h arrays in the hash counter; if not present, update directly if the EFP counter is not full. If the EFP counter is full, update only if the value to be updated is greater than the minimum value in the EFP counter.

3.2. Extensible channel-wise feature group

The feature group describes the upstream, downstream and global context, first-order differencing of packet length distributions, and extensible general features. The data link Layer primarily handles

Algorithm 1 The process of updating parameters such as δ and n to implement the small flow sampling algorithm, where the $.gc()$ function refers to the implicit clearing of counters mentioned earlier.

```
procedure ON_PKT_RECEIVED(f_up, f_down, ts)
    if f_up or f_down in non_target then
        return
    end if
    flow_key ← choose_key(f_up, f_down)
    f_key ← hash(flow_key)
    for i ← 0 to h − 1 do
        cur_item ← hash_counter[i][hash(f_key, i) mod w]
        if cur_item.hashkey = f_key or cur_item.n = 0 then
            cur_item.hashkey ← f_key
            cur_item.n ← (cur_item.n + 1)
            τ_t ← 0
            if cur_item.last_ts ≠ 0 then
                τ_t ← (ts − cur_item.last_ts)
                cur_item.d ← (cur_item.d + τ_t)
            end if
            l_b ← damped_holt.forecast(τ_t)
            δ ← (cur_item.last_delta + l_b)
            if τ_t > δ and cur_item.n > mouse_length then
                non_target.add(flow_key)
                cur_item.reset()
                efp_counter.remove(flow_key)
                return
            else
                cur_item.last_delta ← (cur_item.last_delta − 1_000)   ▷ With a probability of 1 − (p_1)^d.
            end if
            cur_item.last_ts ← ts
        else
            cur_item.n ← (cur_item.n − 1) ▷ With a probability of (p_2)^n.
        end if
    end for
    if flow_key in efp_counter then
        m ← efp_counter.get(flow_key)
        if m > elephant_length then
            non_target.update()
            hash_counter.gc()
            efp_counter.gc()
        else
            efp_counter.update(flow_key)
        end if
    else
        efp_counter.add(flow_key)   ▷ Based on R0.
    end if
end procedure
```

communication within local networks, providing information mainly related to the physical connection and data frame transmission in local networks. It does not contain detailed information about global Internet transmission. Therefore, we did not select any traffic features based on the data link layer. Fields such as IP and port were not included in our designed feature group because they contain a lot of noise unrelated to traffic characteristics and do not directly reflect traffic behavior patterns, which could negatively impact the model. Since all Tor traffic is TCP, we filtered the data to include only TCP traffic.

Based on the characteristics of Tor traffic during the establishment of communication connections and encrypted data transmission phases as shown in Fig. 4, we proposed the feature group for the upstream, downstream and global context, and first-order differencing of packet length distributions. Then, combining the classic statistical features proposed in existing encrypted traffic classification research that have

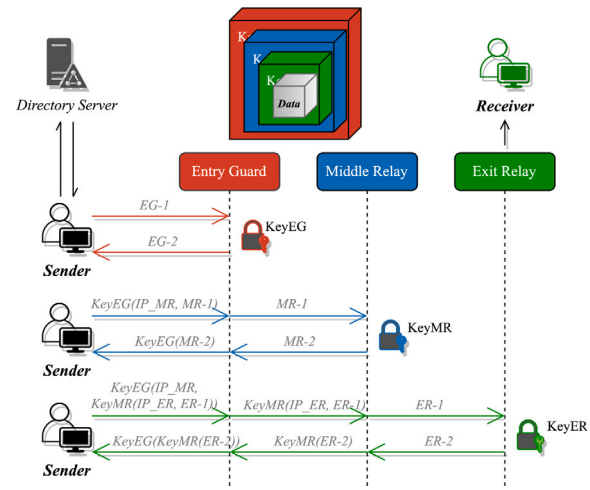

Fig. 4. The behavior of Tor connections includes multi-layer encryption, random selection of relay nodes, and regular path changes.

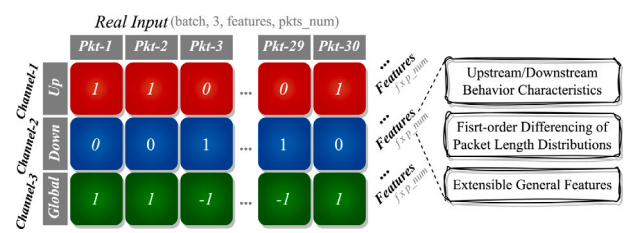

Fig. 5. The characteristics of upstream and downstream traffic in Tor are represented, where UP, DOWN, and GLOBAL denote the three channels of input features. Each channel has a width corresponding to the number of packets and a height representing the different features.

some similarity to the Tor communication process, we proposed an extensible general traffic feature group of classic encrypted traffic features related to darknet traffic. To enable the detection model to better capture the upstream, downstream, and global context features of Tor traffic, this paper treats the upstream features, downstream features, and global context features as channel dimensions of the model input (similar to RGB channels in image processing), thereby better utilizing the proposed detection algorithm to capture the potential relationships among different connection behaviors. By finding appropriate parameters to characterize this feature, we can significantly enhance the overall ability of the model to detect Tor traffic.

3.2.1. Upstream, downstream and global context

In the actual initial model input features, the width dimension represents the number of packets captured in a single flow, and the height dimension indicates the number of different feature representations adopted, as shown in Fig. 5. Taking the upstream, downstream and global context as an example, the input feature shape is (batch, channel, features, pkts_num), and the length of it is the number of packets in the current flow, i.e., 20 or 30. For the upstream channel (Up in the figure), all positions of the upstream packets are set to 1, and the rest are set to 0. The downstream channel (Down in the figure) is the opposite of the upstream channel, and for the global context channel (Global in the figure), upstream packets are set to 1, and downstream packets are set to −1. As shown in Fig. 4, the workflow of Tor traffic can be divided into the following steps: initializing relevant parameters, establishing keys KeyEG, KeyMR, KeyER, and Tor encrypted data transmission.

In the first step, initializing relevant parameters, the *Sender* first obtains the IP addresses of the Entry Guard (EG), Middle Relay (MR), and Exit Relay (ER) nodes from the Tor directory server.

In the second step, establishing keys KeyEG, KeyMR, KeyER, the *Sender* and the EG node use the Diffie–Hellman algorithm (Diffie and Hellman, 2021) to exchange keys for establishing KeyEG. The *Sender* sends key negotiation parameter EG-1 to EG, EG generates another negotiation parameter EG-2 randomly and a symmetric key KeyEG, then sends EG-2 back to the *Sender*. The *Sender* calculates KeyEG using EG-1 and EG-2. To establish KeyMR, the *Sender* uses KeyEG to encrypt the IP address of MR, IP_MR, and the negotiation parameter MR-1. EG decrypts this message using KeyEG to get the IP address IP_MR and MR-1, then sends MR-1 to MR based on the decrypted IP address. MR generates KeyMR using MR-1 and sends its negotiation parameter MR-2 to EG. EG encrypts MR-2 with KeyEG and sends it back to the *Sender*, who then calculates KeyMR using MR-2 and the previous negotiation parameter. To establish KeyER, the *Sender* first encrypts the IP address of ER, IP_ER, and negotiation parameter ER-1 with KeyMR to get KeyMR(IP_ER, ER-1), then encrypts IP_MR and KeyMR(IP_ER, ER-1) with KeyEG to get KeyEG(IP_MR, KeyMR(IP_ER, ER-1)), and sends it to EG. EG decrypts it using KeyEG to obtain IP_MR and KeyMR(IP_ER, ER-1), then sends KeyMR(IP_ER, ER-1) to MR based on IP_MR. MR decrypts it using KeyMR to obtain IP_ER and ER-1, and sends ER-1 to ER. ER generates KeyER and sends the negotiation parameter ER-2 to MR, who encrypts ER-2 with KeyMR and sends it back to EG. EG encrypts KeyMR(ER-2) with KeyEG and sends it to the *Sender*, who then decrypts KeyEG(KeyMR(ER-2)) using KeyEG and KeyMR to obtain ER-2 and calculates KeyER.

The final step is data encryption and transmission, where the *Sender* encrypts *Data* sequentially with KeyER, KeyMR, and KeyEG to obtain KeyEG(KeyMR(KeyER(Data))), then sends it to EG for decryption to obtain KeyMR(KeyER(Data)). EG forwards it to MR, who decrypts it to obtain KeyER(Data) and forwards it to ER, who decrypts it to obtain the final *Data* and sends it to the *Receiver*.

From the analysis of the connection establishment and data transmission process of Tor traffic, the behavioral characteristics of Tor show significant differences compared to ordinary encrypted traffic. Specifically, the multiple key exchanges that occur during connection establishment exhibit distinct upstream, downstream and global context. Additionally, Tor traffic shows significant packet length distribution characteristics when initially accessing node directories and subsequently encrypting data with the three Key layers in an onion-like manner. Compared to other flows, in the same network environment, Tor traffic shows relatively noticeable delays in access time and burst characteristics, such as continuous packet sequences in the same direction, which can be learned by the model through upstream, downstream and global context.

3.2.2. First-order differencing of packet length distributions

The length of the first-order differencing of packet length distributions remains the number of packets in the current flow. For the upstream channel, the value of the position where each upstream packet is located is set to the packet size, and the remaining positions are set to 0. The downstream channel is the opposite of the upstream channel. For the global context channel, upstream packets are set to the packet length, and downstream packets are set to the negative of the packet length.

To strike a balance between detection accuracy and inference speed, we opt to analyze the first-order difference of a specific number of packet lengths at the forefront of a flow, rather than examining the entire sequence of packet lengths or inter-arrival times (IATs) throughout the flow. Furthermore, relying solely on time-related features may not yield accurate results, as the effects of Maximum Transmission Unit (MTU) segmentation can cause packets transmitted earlier in physical time to arrive later at the target due to various network dynamics, including bandwidth fluctuations, topology changes, delays, jitter, congestion control, and routing modifications. Encrypted network traffic naturally exhibits uneven packet transmission intervals, so overemphasizing timing information and using time-related statistical features has its drawbacks. This approach ensures detection accuracy while significantly reducing the processing delay and memory consumption during the feature extraction phase.

According to the Tor data transmission process, it can be observed that the data is forwarded, encrypted, and decrypted through three nodes. To prevent encrypted traffic analysis detection, the Tor protocol typically pads packets to a fixed size, resulting in relatively uniform actual transmission packet sizes, which reduces the possibility of traffic analysis through traditional packet size. This paper examines the upstream, downstream, and global contexts in isolation, correlating the first-order differencing of length characteristics of individual packets with the temporal sequence positions of the respective context vectors. Through the proposed detection algorithm, we endeavor to elucidate the latent first-order differencing of packet length distributions across these three dimensions.

Additionally, by performing certain channel shuffling and weight allocation operations on the extracted feature representations from the middle network layers, we obtain the potential relationship feature representations between the channels. From another subjective analysis perspective, padding packets to a fixed size inherently adds some potential features that cannot be represented by existing classical packet length statistical features.

3.2.3. Extensible general features

The data representation of the extensible general traffic features follows the same principle as the channel-wise feature characteristics. The core lies in the upstream, downstream, and global context features occupying one of the three channels, respectively. The proposed feature representation method can be applied to other application scenarios, and the length of each channel can be adjusted according to the task. Additionally, the height dimension of each channel is a flexible dimension that can be expanded with features such as empty packets, timestamps, and IAT (which may require padding) as needed.

The proposal of the extensible general traffic features gives our method the versatility to handle other types of network traffic detection and classification tasks. The extensibility is reflected in the fact that this paper targets the detection of darknet Tor traffic. Therefore, the proposed features are based on the analysis of Tor traffic's communication mode, and the features that best reflects its communication behavior characteristics are selected to construct the model input. When constructing the model input, we fully considered its extensibility and feature generality. The channel dimension is fixed at 3 dimensions, but the width and height can be increased or decreased according to different task requirements. Almost all network traffic, whether encrypted or not, can have upstream, downstream, and global context dimensions. Therefore, the method proposed in this paper can demonstrate its versatility and rationality. The features used in existing research are listed in Table 1 as a reference for extensible features.

3.3. Model construction

In this section, we introduce a new lightweight CNN model, designed based on operator fusion and channel alignment strategies for detecting darknet traffic. In addition to detecting and classifying traffic, network devices must also provide routine data packet forwarding services. Therefore, any traffic detection and classification methods deployed on the device must not consume excessive device resources. This necessitates that the detection model and its inputs be as lightweight as possible. Therefore, the design goals for the network layers are to select specific types of layers wherever possible and to appropriately order these specific types of layers within the model to increase the potential possibility of more operator fusion and channel alignment. Moreover, the proposed detection algorithm's architecture follows the basic design principles of the latest mainstream lightweight CNN architectures, avoiding operations that increase the model's MAC, and

Table 1
The feature sets proposed in the existing encrypted traffic classification (ETC) methods and their respective application scenarios.

Source	Task	Features
LEXNet	LightweightETC	The size and direction of the first 20 packets.
MTC	ETC	Use a single packet as input, removing the physical layer and redundant information, with a fixed length of 1500 bytes (Ethernet MTU).
NetTiSA	RealTimeETC	Set an active timeout of 300 s and an inactive timeout of 65 s when extracting traffic. Pad insufficient data features with zeros. The categorical features are divided into three groups: bidirectional flow information, statistical and time series analysis features, and features calculated from the first two groups.
NetMamba	PreTrainingETC	Segment packets into flows, selecting the first M packets from each flow, standardizing the header and payload lengths. Use time series analysis methods to cut them into smaller segments, synchronizing the basic temporal information of the network traffic to reduce memory consumption.
Angelo	ETC	Use features such as the number of exchanged packets, data volume, packet rate, bit rate, packet arrival interval time, and average packet length for network traffic classification.
IBGC	ETC	Abstract the packets as interactive actions, grouping them into slices based on direction, and represent the interaction process using a graph structure. The vertices in the graph represent interactive actions, while the edges indicate different transition states. Both vertices and edges can carry additional feature information, such as status codes and packet lengths.
FlowMFD	TorETC	Utilize quantity-frequency-direction spectral features and spatiotemporal modeling to analyze the time series characteristics of packets of different sizes and directions within the flow, extracting long-term temporal patterns.
SPPNet	RealTimeETC	Process the packet header and payload separately for packet-level classification.
FastTraffic	LightweightETC	To improve processing speed, set the IP packet as the granularity, extracting the information part (50 bytes) as input. Use a text-style packet tokenization method and propose an N-gram feature embedding method to represent the original byte structure and sequence features, classifying packets through flow-level labels.

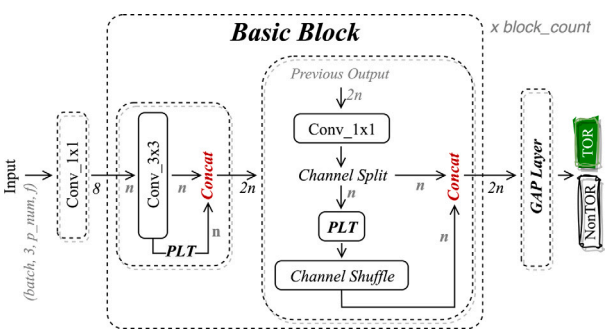

Fig. 6. The main architecture of the lightweight detection algorithm proposed in this paper, which utilizes PLT, channel alignment strategies, MAC reduction by using concatenation (concat), and operator fusion oriented design.

designs that only slightly improve performance but significantly reduce actual inference speed.

To fully leverage operator fusion mechanisms and channel alignment strategies, while ensuring stable detection accuracy and minimizing model parameters, FLOPs, and MAC as much as possible, we proposed a novel lightweight CNN designed with operator fusion and channel alignment strategies in mind. This design draws inspiration from the principles of ShuffleNet (Zhang et al., 2018; Ma et al., 2018), GhostNet (Han et al., 2020), MobileNet (Howard et al., 2017; Sandler et al., 2018; Howard et al., 2019), etc. Typically, when designing CNN architectures, we borrow the residual learning and skip connection ideas from ResNet residual blocks to mitigate gradient vanishing and exploding problems, improve network convergence speed, and enable the model to generalize more easily to new datasets. However, in the context of lightweight network design, the standardized ResNet presents issues such as excessive parameter counts, high computational demands, and difficulty in maintaining lightweight characteristics. Therefore, we have made relevant trade-offs and optimizations according to the needs of this task. Existing lightweight encrypted traffic detection and classification models primarily focus on reducing model parameters and FLOPs, neglecting the significant impact of MAC on actual inference speed. Besides general matrix-matrix multiplication (GEMM), the GPU is relatively slow at handling other operations. For instance, although normalization and pointwise operations have much lower FLOPs than GEMM, non-GEMM operations consume much more runtime than they should due to MAC.

The proposed detection algorithm's overall architecture consists of four main parts: the input shallow feature extraction layer, PLT & concat operation, channel alignment and operator fusion strategies, and Global Average Pooling (GAP) layer. The core design principles of this lightweight detection algorithm are as follows:

(I) Use PLT to replace traditional convolution operations, significantly reducing FLOPs and the number of parameters.
(II) Employ a channel alignment strategies to align the dimensions of input/output tensors with a certain minimum granularity (vary depending on the device), allowing the tensor core layers to achieve better performance.
(III) Use the concat operation at branch merging points to reduce memory access times.
(IV) Arrange the network structure based on operator fusion strategies to maximize the utilization of CUDA cores.

First, as shown in the leftmost part of Fig. 6, it uses a basic 1×1 convolution operation for initial shallow feature extraction. After completing the shallow feature extraction of the initial input feature matrix, the basic block is used for deeper potential feature learning and, more importantly, the potential relationships between different channels. Based on our channel dimension design and the setting of input features, this means that the model can learn the potential feature representations among upstream packets, downstream packets, between upstream and downstream packets, and between upstream, downstream packets, and the global context. After passing through multiple basic blocks, several matrices containing local and global feature relationships are generated. Based on these feature matrices, the GAP replaces the traditional fully connected layer (FC) to detect Tor traffic (as shown in the rightmost part of Fig. 6).

The initial input data consists of Tor traffic feature matrices that have undergone rule-based preprocessing and small flow sampling, represented as a tensor with the shape $(batch, 3, N_s, N_f)$. N_s represents

the number of packets selected per flow, N_f represents the number of feature dimensions selected. These latter two values can theoretically be configured differently according to the requirements of different tasks for accuracy, real-time performance, and hardware conditions, to meet the needs of the corresponding tasks.

3.3.1. PLT and concat operations

As shown in Fig. 6, the PLT & concat operations is composed of two convolution operations with the same number of input and output channels and a concat operation. One of these convolution operations is a 3×3 convolution operation, and the other is a simple linear transformation operation. Typically, convolution operations are used to increase the number of channels to enhance the capability of extracting potential features, but this leads to a surge in the number of parameters, FLOPs, and MAC.

According to the algorithm design principles I proposed in this article, we increase the number of feature map channels by performing simple linear transformation operations on the output feature maps of the convolution operations. These linearly transformed feature maps are concatenated with the original convolution output feature maps. The simple linear transformation operation is a convolution operation with the same number of input and output channels, a kernel size of 3×3, a stride of 1, padding of 1, and the number of groups equal to the number of channels. This operation performs convolution on each input channel separately and outputs the same number of channels, significantly reducing the computational load.

According to the algorithm design principles III, using the concat operation to splice feature maps between different layers retains more contextual information. For such multi-branch merging scenarios, TensorRT can directly connect to the required locations without performing a dedicated concat operation. Therefore, when our model uses the concat operation instead of element-wise operations like Add, this layer will eventually be merged with the next operation using the same CUDA core, resulting in fewer computational graph layers and fewer occupied CUDA cores.

3.3.2. Channel alignment and operator fusion strategies

As shown in Fig. 6, to reduce computational load, the channel alignment & operator fusion incorporates a 1×1 convolution operation before the PLT operation to facilitate the flow of information between global channels. This is because PLT operations cannot exchange information between feature representations of different groups. Then, the input feature representations with n channels are split into two branches with c and $n-c$ channels ($c = n$, in this article), respectively. Following ResNet's skip connection idea, the branch with $n-c$ channels is left unprocessed, while the other branch with c channels undergoes a PLT operation with c channels and a kernel size of 3×3.

According to the channel alignment design guideline II, when the number of input and output channels for each convolutional operation within a residual block is the same, and the channel count for convolutional operations both within and outside the block is a multiple of 8, the tensor dimensions for GEMM, fully connected layers, convolution, and deconvolution can be aligned to the smallest granularity to achieve better performance. When TensorRT optimizes model inference, it has a padding strategy that rounds up tensor dimensions to the nearest multiple of 8, allowing the model to utilize additional performance capacity without increasing computation or memory consumption. Therefore, if the number of channels is not a multiple of this value, some performance will be wasted. Conversely, this can minimize MAC and genuinely reduce the actual inference time of the model. Then, channel shuffle operations are performed to ensure information exchange within this branch. According to design guideline III, we use concat splicing to reduce the CUDA core occupation during the final inference stage.

The final output layer replaces the traditional FC layer with GAP to map the feature vectors generated by the basic blocks to network traffic categories, indicating whether it is darknet traffic. The FC layer usually contains a large number of parameters, which can lead to overfitting problems. Using GAP can significantly reduce the number of parameters, making the model more robust and better at resisting overfitting.

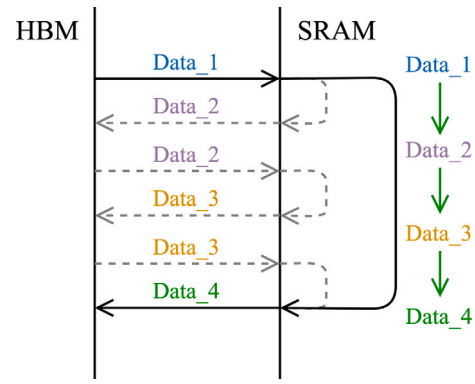

Fig. 7. Example of reducing data transfers between High Bandwidth Memory (HBM) and Static Random Access Memory (SRAM).

3.3.3. Analysis of reducing data transfers

According to the mode fusion design guideline IV, Conv and Scale operations can be merged into one operator, and Scale can be merged with Activation operators. Therefore, our convolution operations are followed by BN and ReLU operations, enabling the combination of three operations into a single CUDA core, significantly reducing MAC.

The access rate of SRAM is about 10 times that of HBM, but the amount of data it can carry is far less than HBM. Excessive access to HBM, such as accessing data immediately after storing it in HBM, leads to limited algorithm performance and high memory usage due to HBM's low bandwidth, as shown in Fig. 7. To alleviate this issue, the inference process can be optimized through operator fusion. Operator fusion is a process that combines different operators into a single computation, usually fusing point-wise operators into reduction and GEMM, and GEMM itself can be considered a fusion of broadcasting multiplication and reduction operations. These different basic operators form the foundational layers we commonly see in neural network structure design, such as convolution, BN, and activation layers. Fusion can simplify the network while maintaining the same overall behavior. TensorRT supports custom operators to achieve more advanced operator fusion, but this is a very time-consuming development process. The goal of this lightweight detection algorithm is to fully utilize the operators already supported to design a reasonable network structure that maximizes the fusion of network layers, reducing the time-consuming and inefficient HBM-to-SRAM data transfers. For instance, the internal implementation of convolution supports directly computing the output of the ReLU function from the convolution kernel without a second kernel call.

4. Experiments

In this chapter, we first describe the public datasets used, the synthetic low-proportion darknet dataset and the data collection methods for the synthetic dataset. We then outline the key experimental settings, including the design of the detection algorithm experiments and the comparison experiments. Additionally, we provide detailed information on the hardware parameters and software environment used in the experiments.

4.1. Dataset

To comprehensively assess the efficacy of our proposed methodology, we leveraged a plethora of publicly available traffic datasets and constructed a synthetic low-proportion darknet traffic dataset, thereby facilitating a rigorous evaluation of our approach. These datasets were selected to cover different types of network environments and traffic patterns, ensuring the broad applicability and reliability of the experimental results. The traffic data sources, collection methods, and characteristics of each dataset are detailed below. Considering the extremely low-proportion of Tor traffic in backbone networks, we simulated real scenarios by mixing public backbone network traffic datasets with different proportions of Tor traffic.

- **The MAWI dataset** (Cho, 2017) originates from the daily network traffic capture of the WIDE project in Japan. This dataset captures network traffic from a backbone network between Japan and the United States at different time periods (e.g., a specific minute each day, each month), involving a total of seven sampling points with a wide time span (from 2001 to 2018), providing real network usage scenarios.
- **The VANT dataset** (Jorgensen et al., 2023) is a labeled collection of PCAP files, including encrypted and unencrypted traffic from 10 applications. The dataset size is 36.1 GB, containing 33,711 connections, approximately 272 h of traffic capture, covering five traffic categories: Streaming, VoIP, Chat, Command & Control (C2), and File Transfer.
- **The CIC-Darknet2020 dataset** (Lashkari et al., 2020) was published by the Canadian Institute for Cybersecurity (CIC), which is specifically designed for the study of darknet traffic. The dataset includes various types of darknet traffic and normal network traffic, categorized into eight types: Audio-Stream, Browsing, Chat, Email, P2P, Transfer, Video-Stream, and VOIP.
- **The TCUB dataset** (Hu et al., 2020) contains user behavior traffic in Tor, I2P, ZeroNet, and FreeNet. It is divided into 8 categories: Browsing, Chat, E-mail, Audio-streaming, Video-streaming, File Transfer, P2P, and VoIP, similar to the CIC-Darknet2020 dataset; however, the latter only contains Tor traffic.
- **The self-built low-proportion darknet dataset** is synthesized by capturing Tor traffic in a simulated network environment, along with non-Tor traffic from the TCUB and VANT public datasets, as well as backbone network traffic from the MAWI public dataset. This process results in different proportions of darknet traffic. Additionally, we replace our self-built Tor traffic source with Tor traffic from the CIC-Darknet2020 and TCUB public datasets to synthesize a low-proportion dataset with varying darknet traffic ratios using the aforementioned method.

In the process of collecting self-built low-proportion Tor traffic data, we deployed multiple traffic capture devices across various Tor nodes. It is noteworthy that the monitoring occurs between the client and the entry guard, as a substantial amount of raw traffic from users can be easily captured at this stage. These devices were configured to simulate the transmission and collection of authentic Tor traffic data within the same virtual Ethernet (veth) pair, thereby ensuring that the captured data exclusively comprised Tor traffic. Over a duration of a considerable period of time, these devices were capable of capturing all incoming and outgoing traffic data, providing a sufficiently extensive temporal coverage of the dataset.

Notably, our approach involved the deployment of multiple Tor clients across servers situated in diverse regional locations, enabling us to establish connections with numerous Tor nodes. After these Tor clients establish connections to the darknet, we run preconfigured Bash scripts to simulate accesses to different categories of resources (e.g., browsing web pages, streaming audio/video, etc.) in order to mimic genuine user behavior. Consequently, the Tor traffic capture occurs between the client and the entry guard. To ensure that the traffic capture device solely records all traffic emanating from the Tor client, we employ Docker[5] containers to run the Tor client. When Docker creates a container exclusively for the Tor client service, it establishes a separate network namespace for the container, which includes container-specific network interfaces and routing tables. Docker subsequently generates a veth pair, consisting of two interconnected interfaces: data sent from one interface appears immediately on the other. One end of the veth pair is allocated to the container's network namespace, while the other is assigned to the host's network namespace. By capturing traffic on the uniquely identified network interface associated with the container's end of the veth pair, we effectively avoid potential data contamination.

In real-world backbone network level environments, Tor traffic typically constitutes a negligible proportion of the overall traffic. To replicate this condition, we integrated the MAWI and VANT public network traffic datasets, varying the proportions of Tor traffic for our experiments. Specifically, we conducted experiments with low-proportion Tor traffic representing 0.01%, 0.05%, 0.1%, 0.5%, 1%, 5%, and 10% of the total traffic. These proportions reflect potential fluctuations in Tor traffic across diverse networks and temporal contexts. Notably, even at a Tor traffic proportion as low as 0.01%, which closely mirrors the conditions found in backbone networks, our methodology demonstrated a stable detection accuracy, thereby affirming the robustness and reliability of our proposed approach.

4.2. Evaluation setting

This section is designed to evaluate the performance of our proposed lightweight algorithm for Tor traffic detection. By designing collaborative filtering comparative experiments, multiple sets of feature comparison experiments, comparative experiments of different models on our dataset and model structure ablation experiments, we aim to comprehensively analyze the model's performance under different conditions and verify the superiority and robustness of the designed feature sets and the proposed algorithms.

4.2.1. Collaborative filtering comparison experiments

Comparison of results with and without collaborative filtering, different sampling ratios and tor source datasets. We will train and test these feature sets under the same model architecture and compare their detection accuracy and model inference speed.

4.2.2. Extensible channel-wise feature effectiveness experiments

To investigate the effectiveness of the proposed channel-wise extensible features, we established baseline comparative experiments using the same feature sources, as well as experiments comparing the performance of our proposed method with and without the channel-wise extensible features across different datasets.

4.2.3. Tor detection algorithm experiments

We set up experiments to measure the actual inference speed of the detection model. In addition, to verify the validity of each component in our model, we have designed the following ablation experiments: (1) The impact of removing the design guidelines; (2) The comparison of different models on our dataset.

[5] https://docs.docker.com/engine/network/.

Table 2
Hardware and software configuration information used in the experiments.

Attribute	Value
Device	NVIDIA GeForce RTX 3060
Memory	11927 MiB
Memory clock rate	7.501 GHz
Memory bus width	192.0
SMs	28.0
Cuda version	12.6.85
TensorRT version	10.7.0
PyTorch version	2.5.1

4.3. Model training and hyperparameters

In this paper, the hardware and software configurations employed during the experiments are delineated in Table 2. Specifically, the experiments were conducted on a system equipped with an NVIDIA GeForce RTX 3060 graphics card, featuring 11 927 MiB of memory and a memory clock rate of 7.501 GHz. The memory bus width is 192.0, and the system comprises 28 streaming multiprocessors (SMs). The CUDA version utilized is 12.6.85, alongside TensorRT version 10.7.0 and PyTorch version 2.5.1. Furthermore, to facilitate data processing and visualization, we employed a suite of prevalent libraries, including Numpy, Scipy, Pandas, and Matplotlib.

To train an effective model for detecting Tor encrypted traffic, we selected a set of optimized training parameters that demonstrated good performance and stability in preliminary experiments. We first preprocess the data by dividing the Tor traffic dataset into training, validation, and test sets, which are used respectively for model training, parameter tuning, and performance evaluation. During the model training process, we adopt the Adam optimization algorithm. To prevent overfitting, we use two regularization techniques during training: L2 regularization and Dropout. The training process includes forward propagation, loss calculation, backpropagation, and parameter updates. At the end of each training epoch, we evaluate the model's performance using the validation set and adjust the model parameters based on the validation results.

Our network layer design prioritizes operator fusion and channel alignment strategies as we transition into the model inference optimization phase. In this phase, we utilize TensorRT to convert and optimize the trained model for improved inference efficiency. The model, originally trained in a deep learning framework, is exported in ONNX format. During the engine build phase, TensorRT automatically detects and fuses operators, effectively reducing computational overhead. The result is an optimized inference engine capable of rapidly detecting Tor encrypted traffic.

5. Results and discussions

This study aims to enhance the monitoring and analysis capabilities of low-proportion darknet traffic through efficient collaborative filtering and the lightweight darknet traffic detection algorithm. Various visualization techniques were employed to clearly observe the model's detection accuracy steadily improving. Furthermore, tables illustrate the significant performance improvement with the introduction of channel-wised features (upstream, downstream, and global context), proving the importance of this design in enhancing the proposed algorithm's detection capabilities. The application of collaborative filtering technology plays a crucial role in detecting extremely low-proportion darknet traffic in backbone networks. Finally, we conducted a comprehensive analysis of the model ablation experiment results, discussing the effectiveness of the proposed design guidelines and the role these specific designs play in maximizing operator fusion and channel alignment strategies.

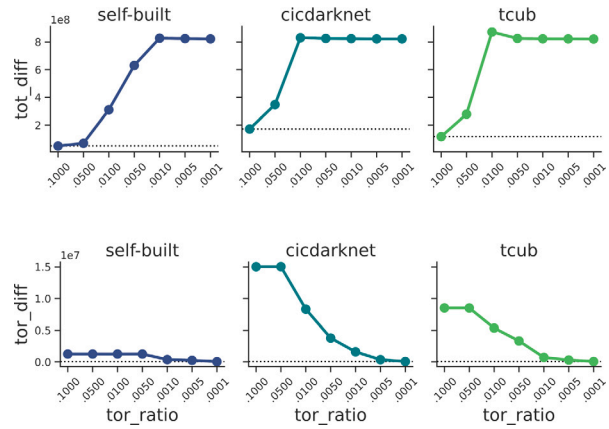

Fig. 8. The difference in total and tor packet count before and after collaborative filtering for different Tor low-proportion datasets.

5.1. Experiments on collaborative filtering

Packets & Flows As illustrated in Tables 3 and 4, this study utilizes a self-constructed Tor traffic dataset, along with the CIC-Darknet2020 and TCUB datasets, as sources of darknet traffic. Additionally, the MAWI and VANT public datasets were configured as background traffic to create low-proportion darknet datasets. The total number of packets, total flows, total Tor packets, and total Tor flows were recorded at proportions of .1000, .0500, .0100, .0050, .0010, .0005, and .0001. It is evident that in all three low-proportion datasets, as the proportion of darknet traffic decreases, the volume of data requiring processing surges dramatically, reaching a peak of **998.4M packets** and **63.3M flows**.

Rule-Based Preprocessing To validate the effectiveness of the rule-based preprocessing, we conducted a statistical analysis of the total number of flows and packets before and after the application of the rule-based approach. As illustrated in Table 7, the terms "+UDP" and "+Other" represent TCP plus UDP and other additional IP-based protocols. Conversely, "R-TCP" and "R-Tor" represent the flow counts after the rule-based preprocessing, and these preprocessed flows are all based on the TCP protocol. Combining Tables 4 and 7, it is evident that the rule-based preprocessing ensures that no flows are lost. Furthermore, by examining the "total" column from each low-proportion darknet datasets in Table 3 alongside the data in Table 8, it becomes apparent that the rule-based preprocessing substantially diminishes the volume of data necessitating processing.

Packets Reduction Ratios In the three low-proportion darknet datasets presented in Table 4, the number of Tor traffic flows remained intact following the application of collaborative filtering, ensuring that no single Tor flow was lost. Concurrently, as shown in Tables 5, 6 and Fig. 8, our collaborative filtering algorithm effectively filtered out **89.38%** of packets in the self-built low-proportion darknet dataset at a .0010 ratio, with an initial timeout threshold of 8 s. In the CIC-Darknet low-proportion dataset at a .1000 ratio, the algorithm achieved a filtering rate of **91.26%**, while in the TCUB low-proportion dataset at a .0050 ratio, it filtered out **89.79%** of packets under the same timeout condition. Due to the relatively low volume of Tor darknet packets, the proportion of data reduction achieved is significantly greater than that of non-Tor packets, as shown in Table 6. However, our approach utilizes a minimal number of packets (for instance, 20 or 30) to detect early connection behaviors associated with darknet traffic. As long as the connection process can be detected, the subsequent transmission phase, which involves a larger volume of packets, becomes irrelevant.

Initial Timeout Threshold We employed a dynamic timeout decay strategy to expedite the collaborative filtering process, while distinguishing between new and old flows within the same five-tuple (with

Table 3
The statistical analysis of packet quantities across various low-proportion darknet datasets is presented, where "tor" signifies the number of packets containing Tor traffic within each dataset, and "total" represents the overall packet count corresponding to those datasets. The first column delineates the anticipated proportion of Tor traffic within the synthesized low-proportion datasets, while the "real ratio" reflects the authentic proportion derived from the experimental environment.

	self-built			cicdarknet			tcub		
	tor	total	real ratio	tor	total	real ratio	tor	total	real ratio
.1000	1.3M	59.3M	.0215	15.0M	196.5M	.0765	8.5M	146.9M	.0581
.0500	1.3M	84.6M	.0150	15.0M	416.3M	.0361	8.5M	339.3M	.0252
.0100	1.3M	379.4M	.0033	698.3M	950.4M	.0088	5.4M	**998.4M**	.0054
.0050	1.3M	725.4M	.0018	3.8M	945.1M	.0040	3.3M	944.9M	.0035
.0010	380.1K	946.8M	.0004	1.6M	943.5M	.0017	726.0K	942.4M	.0008
.0005	265.3K	944.2M	.0003	364.4K	942.3M	.0004	318.3K	942.5M	.0003
.0001	51.5K	942.3M	.0001	72.9K	942.0M	.0001	73.4K	941.0M	.0001

Table 4
The quantity of Tor and total flows.

	self-built		cicdarknet		tcub	
	tor	total	tor	total	tor	total
.1000	1019	4.7M	117	10.9M	1925	14.2M
.0500	1005	7.2M	117	33.5M	1920	29.6M
.0100	1016	26.8M	69	63.3M	1806	63.3M
.0050	1019	46.9M	20	63.3M	1721	63.3M
.0010	354	63.3M	8	63.3M	532	63.3M
.0005	199	63.3M	4	63.3M	485	63.3M
.0001	39	63.3M	2	63.3M	11	63.3M

upstream and downstream flows treated as a single flow) to further reduce the number of packets requiring redundant processing. As evidenced by the experimental results in Tables 5 and 6, in all three low-proportion darknet datasets, a decrease in the initial timeout threshold correlates with an increase in the proportion of filtered traffic packets (both total and Tor), with the maximum increase reaching **5.43%**.

5.2. Performance on extensible channel-wise features

The channel-wise traffic features (upstream, downstream, and global context) simulate the three-channel model commonly used in image processing. This design aims to capture the latent connection behavior characteristics of darknet traffic while enhancing the applicability of feature representation through its extensibility. To validate the effectiveness of the extensible channel-wise features, we modified the input size of the proposed Tor detection model to match that of the baseline model, LEXNet, with a size of (batch, 1, 20, 2). Given the extensibility of our feature representation, this adjustment was straightforward. We then compared the results by incorporating scalable Inter-Arrival Time (IAT) and Inter-Packet Delay (IPD) features alongside the same upstream and downstream packet lengths used in the baseline model, distinguishing between scenarios with and without channel-wise processing.

Channel-Wise The experimental results, as shown in Table 9, indicate that while the baseline model successfully detects darknet Tor traffic, our proposed model, utilizing identical feature inputs, also achieves effective detection with performance closely matching that of the baseline. Specifically, the baseline model and our model yielded results of 93.77% and 93.61%, respectively, on the self-built dataset; 97.89% and 82.17% on the CIC-Darknet2020 dataset; and 89.27% and 81.52% on the TCUB dataset. Notably, after incorporating channel-wise processing, our model's detection performance improved significantly across all three datasets, reaching **99.77%**, **97.45%**, and **98.19%**, respectively.

Extensibility The experimental results corresponding to the IAT and IPD columns in Table 9 indicate that the extended IAT and IPD features, when not processed using channel-wise, exhibit varying detection outcomes compared to both the baseline model and our model utilizing identical inputs. However, both approaches generally maintain

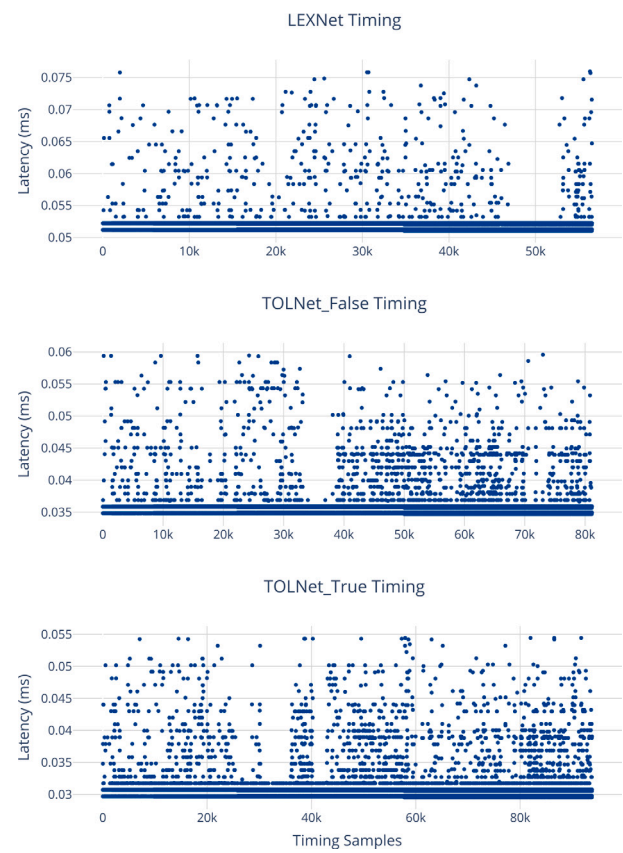

Fig. 9. A comparison of the actual inference speed of our model before and after using operator fusion and channel alignment design with the LEXNet model.

satisfactory detection performance. This suggests that the extensible design is indeed feasible and can accommodate diverse application requirements by extending various generic features or even the raw byte data of packets (though this may compromise real-time processing and is more suitable for other application scenarios). Moreover, the results for IAT and IPD features improved significantly after the implementation of channel-wise processing. Specifically, the detection performance for IAT across the three datasets was **99.46%**, **99.36%**, and **99.64%**, while the detection performance for IPD reached **99.77%**, **99.79%**, and **99.88%**, respectively.

5.3. Performance on model inference efficiency

Operator Fusion & Channel Alignment The network architecture designed for operator fusion and channel alignment strategies is converted to ONNX format after model training, enabling compilation

Table 5
The proportion of total packets filtered out under different proportion and timeout threshold.

	self-built				cicdarknet				tcub			
	8	16	32	64	8	16	32	64	8	16	32	64
.1000	84.86%	83.04%	81.76%	81.57%	**91.26%**	89.15%	87.35%	85.83%	83.46%	81.15%	79.32%	**78.01%**
.0500	83.58%	81.87%	80.34%	**79.69%**	86.08%	84.15%	82.58%	**81.17%**	84.77%	82.81%	81.22%	79.82%
.0100	84.32%	82.47%	80.95%	79.72%	89.48%	88.02%	86.73%	85.49%	89.33%	87.90%	86.63%	85.39%
.0050	88.96%	87.38%	85.99%	84.66%	89.18%	87.78%	86.54%	85.35%	**89.79%**	88.22%	86.84%	85.50%
.0010	**89.38%**	87.94%	86.65%	85.42%	89.27%	87.83%	86.56%	85.34%	89.62%	88.09%	86.72%	85.41%
.0005	89.22%	87.79%	86.55%	85.35%	89.24%	87.79%	86.53%	85.32%	89.25%	87.81%	86.54%	85.33%
.0001	89.19%	87.76%	86.50%	85.31%	89.27%	87.83%	86.56%	85.33%	89.45%	87.95%	86.64%	85.38%

Table 6
The proportion of tor packets filtered out under different proportion and timeout threshold.

	self-built				cicdarknet				tcub			
	8	16	32	64	8	16	32	64	8	16	32	64
.1000	98.62%	98.05%	97.25%	96.91%	**99.99%**	99.98%	99.98%	99.96%	99.72%	99.66%	99.59%	99.56%
.0500	98.79%	98.26%	97.64%	96.87%	**99.99%**	99.98%	99.97%	99.96%	99.73%	99.67%	99.61%	99.56%
.0100	98.95%	98.62%	98.18%	97.73%	**99.99%**	**99.99%**	99.98%	99.98%	99.70%	99.60%	99.52%	99.45%
.0050	**99.12%**	98.75%	98.32%	97.62%	**99.99%**	**99.99%**	**99.99%**	99.98%	99.48%	99.35%	99.25%	99.13%
.0010	99.02%	98.67%	98.40%	97.91%	99.98%	99.97%	99.98%	99.97%	99.30%	99.08%	98.98%	98.85%
.0005	97.66%	97.02%	96.53%	**95.96%**	99.93%	99.94%	99.95%	99.95%	98.90%	98.50%	98.26%	**98.05%**
.0001	98.21%	97.79%	97.08%	95.75%	99.84%	99.84%	**99.83%**	99.84%	98.90%	**99.76%**	99.75%	99.73%

Table 7
The table below depicts the variation in flow count when using rule-based preprocessing exclusively, with "+Other" representing a wide range of non-TCP traffic, including UDP and other IP-based protocols. The R-TCP and R-Tor labels indicate the flow counts after applying rule-based preprocessing.

	self-built				cicdarknet				tcub			
	+Other	+UDP	R-TCP	R-Tor	+Other	+UDP	R-TCP	R-Tor	+Other	+UDP	R-TCP	R-Tor
.1000	12.9M	6.5M	4.7M	**1019**	31.7M	15.4M	10.9M	**117**	38.7M	19.2M	14.2M	**1925**
.0500	19.9M	9.9M	7.2M	**1005**	67.3M	43.2M	33.5M	**117**	62.0M	38.2M	29.6M	**1920**
.0100	58.1M	34.6M	26.8M	**1016**	155.7M	85.0M	63.3M	**69**	155.5M	84.9M	63.3M	**1806**
.0050	106.7M	61.6M	46.9M	**1019**	155.6M	84.9M	63.3M	**20**	155.7M	85.0M	63.3M	**1721**
.0010	155.7M	85.0M	63.3M	**354**	155.7M	85.0M	63.3M	**8**	155.7M	85.0M	63.3M	**532**
.0005	155.7M	85.0M	63.3M	**199**	155.7M	85.0M	63.3M	**4**	155.7M	85.0M	63.3M	**485**
.0001	155.7M	85.0M	63.3M	**39**	155.7M	85.0M	63.3M	**2**	155.5M	84.9M	63.3M	**11**

Table 8
The number of packets corresponding to each low-proportion datasets following only rule-based preprocessing.

	self-built	cicdarknet	tcub
.1000	25.1M	140.2M	79.6M
.0500	38.6M	252.6M	200.4M
.0100	203.3M	574.4M	603.1M
.0050	464.4M	569.4M	569.0M
.0010	566.2M	567.5M	566.4M
.0005	566.2M	566.4M	566.4M
.0001	566.0M	566.0M	565.7M

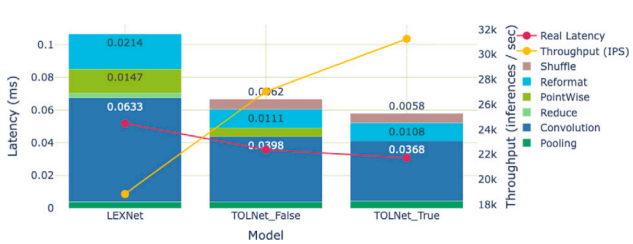

Fig. 10. The actual inference speed and throughput comparison between models. Throughput is calculated by dividing the number of queries by the total host wall time. This wall time is measured from the moment the first query is queued until the last query is finished processing.

into an engine that maximally leverages CUDA cores. Through continuous fine-tuning of the network layers and structure, we achieved superior inference efficiency compared to the baseline model, LEXNet, which, although optimized through engine compilation, did not employ strategies aimed at maximizing CUDA core utilization. As shown in Table 10, our model has an average inference time of **0.058 ms**, while the baseline model takes 0.107 ms, making it **45.79%** slower than ours. Additionally, our model's throughput exceeds that of the baseline by **39.72%**, reflecting the benefits of reduced MAC operations as highlighted in Fig. 7.

Inferencing Speed Analysis In the context of TOLNet, "False" and "True" denote the use and non-use of operator fusion and channel alignment strategies in our proposed model, respectively. Figs. 9 and 10 illustrate the detailed time consumption of each optimized neural network layer during inference for the compiled engines of LEXNet, TOLNet_False and TOLNet_True. Compared to TOLNet_True, both models exhibit higher time consumption across their respective layers, with times concentrated around 0.052 ms and 0.036 ms, while TOLNet_True achieves the lowest time, primarily around **0.030 ms**. Despite TOLNet_False utilizing channel-wise processing and having a larger input dimension, it remains faster than LEXNet. It is worth mentioning that both models have a similar number of network layers. This is attributed to the fact that, although neither model employs strategies for operator fusion and channel alignment, LEXNet incorporates additional Reformatting and PointWise operations for model interpretability, which adds to its computational overhead.

6. Conclusions and future work

In this paper, we propose a novel lightweight detection algorithm, designed based on the strategies of operator fusion and channel alignment. Combined with a collaborative filtering algorithm and our proposed channel-wise upstream and downstream behavior characteristics,

Table 9
The comparison of detection results using channel-wise features versus not using channel-wise features, along with the baseline method. In this paper, the proposed model TOLNet uses an input size of (batch, 1, 20, 2) when not using channel-wise features, which is consistent with the baseline method LEXNet. When using channel-wise features, the input size is (batch, 3, 20, 2). In addition, IPD refers to inter-payload difference.

	Baseline-LEXNet	Channel-wised features		+Expand IAT		+Expand IPD	
	Not used	Used	Not used	Used	Not used	Used	Not used
Self-built	0.937650	**0.997691**	0.936105	**0.994611**	0.881447	**0.997691**	0.916089
cicdarknet	0.978858	**0.974522**	0.821656	**0.993631**	0.895966	**0.997877**	0.951168
tcub	0.892642	**0.981884**	0.815217	**0.996377**	0.962560	**0.998792**	0.915459

Table 10
Comparison of model inference efficiency. Throughput is calculated by dividing the number of queries by the total host wall time. A higher throughput value signifies better performance, while a lower average time indicates greater efficiency.

	LEXNet	TOLNet_False	TOLNet_True
Avg. time	0.107 ms	0.067 ms	**0.058 ms**
Throughput	18 840.9	27 049.5	**31257.4**

packet length distributions, and extensible general traffic features, we achieve a method for detecting extremely low-proportion darknet Tor traffic in backbone networks. By incorporating design principles from lightweight CNNs, we designed a neural network with efficient input feature extraction and traffic detection capabilities. Experimental results demonstrate that the proposed algorithms achieve high detection accuracy comparable to SOTA method while simultaneously reducing computational complexity and inference time. Specifically, our collaborative filtering algorithm is capable of filtering out up to 91.26% of packets without losing any flow, which is critically important for detecting extremely low-proportion dark web traffic in high-speed backbone network scenarios. Furthermore, our experimental findings validate the effectiveness of the extensible channel-wise feature group and their potential for extension to other application contexts. Notably, the actual inference time is reduced by 45.79% compared to SOTA traffic detection models, thereby enabling real-time detection capabilities.

The findings of this study offer valuable insights and potential avenues for future research. Firstly, while the proposed lightweight detection model significantly enhances efficiency, there remains considerable scope for further optimization. For instance, investigating more advanced weight pruning techniques and quantization methods could lead to additional reductions in model size and computational costs. Secondly, future investigations could focus on integrating more contextual information and extended feature sets to enhance traffic detection performance across a broader range of application scenarios. Moreover, a foundational theoretical analysis of the false positive rate associated with collaborative filtering, as well as further optimization of storage space for counters, merits further investigation. With ongoing improvements and refinements, the collaborative filtering algorithm, extensible channel-wise feature group, and the lightweight model design oriented towards operator fusion and channel alignment proposed in this paper are anticipated to play an increasingly pivotal role in the realm of real-world darknet traffic detection.

CRediT authorship contribution statement

Qiang Zhang: Writing – original draft, Conceptualization, Methodology, Software. **Cheng Huang:** Funding acquisition, Investigation, Supervision. **Jiaxuan Han:** Project administration, Visualization. **Shuyi Jiang:** Data curation, Formal analysis. **Jiayong Liu:** Investigation, Resources, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Abdo, Jacques Bou, Hossain, Liaquat, 2023. Modeling the invisible internet. In: International Conference on Complex Networks and their Applications. Springer, pp. 359–370.

Al-Omari, Ahmad, Allhusen, Andrew, Wahbeh, Abdullah, Al-Ramahi, Mohammad, Alsmadi, Izzat, 2022. Dark web analytics: A comparative study of feature selection and prediction algorithms. In: 2022 International Conference on Intelligent Data Science Technologies and Applications. IDSTA, IEEE, pp. 170–175.

Angel, Yawning, Winter, Philipp, 2014. obfs4 (the obfourscator).

Bujlow, Tomasz, Carela-Español, Valentín, Barlet-Ros, Pere, 2015. Independent comparison of popular dpi tools for traffic classification. Comput. Netw. 76, 75–89.

Cheng, Hongrong, Zhang, Miao, Shi, Javen Qinfeng, 2024. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. IEEE Trans. Pattern Anal. Mach. Intell..

Cho, Kenjiro, 2017. Recursive lattice search: Hierarchical heavy hitters revisited. In: Proceedings of the 2017 Internet Measurement Conference. pp. 283–289.

Clarke, Ian, Sandberg, Oskar, Wiley, Brandon, Hong, Theodore W., 2001. Freenet: A distributed anonymous information storage and retrieval system. In: Designing Privacy Enhancing Technologies: International Workshop on Design Issues in Anonymity and Unobservability Berkeley, CA, USA, July 25–26, 2000 Proceedings. Springer, pp. 46–66.

Diffie, Whitfield, Hellman, Martin, 2021. New directions in cryptography (1976).

Dingledine, Roger, Mathewson, Nick, Syverson, Paul F., et al., 2004. Tor: The second-generation onion router. In: USENIX Security Symposium. Vol. 4, pp. 303–320.

Fauvel, Kevin, Chen, Fuxing, Rossi, Dario, 2023. A lightweight, efficient and explainable-by-design convolutional neural network for internet traffic classification. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 4013–4023.

Goldschlag, David, Reed, Michael, Syverson, Paul, 1999. Onion routing. Commun. ACM 42 (2), 39–41.

Gu, Jiuxiang, Wang, Zhenhua, Kuen, Jason, Ma, Lianyang, Shahroudy, Amir, Shuai, Bing, Liu, Ting, Wang, Xingxing, Wang, Gang, Cai, Jianfei, et al., 2018. Recent advances in convolutional neural networks. Pattern Recognit. 77, 354–377.

Han, Kai, Wang, Yunhe, Tian, Qi, Guo, Jianyuan, Xu, Chunjing, Xu, Chang, 2020. Ghostnet: More features from cheap operations. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 1580–1589.

He, Kaiming, Zhang, Xiangyu, Ren, Shaoqing, Sun, Jian, 2016. Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 770–778.

Howard, Andrew, Sandler, Mark, Chu, Grace, Chen, Liang-Chieh, Chen, Bo, Tan, Mingxing, Wang, Weijun, Zhu, Yukun, Pang, Ruoming, Vasudevan, Vijay, et al., 2019. Searching for mobilenetv3. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 1314–1324.

Howard, Andrew G., Zhu, Menglong, Chen, Bo, Kalenichenko, Dmitry, Wang, Weijun, Weyand, Tobias, Andreetto, Marco, Adam, Hartwig, 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.

Hu, Yuzong, Zou, Futai, Li, Linsen, Yi, Ping, 2020. Traffic classification of user behaviors in tor, i2p, zeronet, freenet. In: 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications. TrustCom, IEEE, pp. 418–424.

Huang, Gao, Liu, Shichen, der Maaten, Laurens Van, Weinberger, Kilian Q., 2018. Condensenet: An efficient densenet using learned group convolutions. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 2752–2761.

Jorgensen, Steven, Holodnak, John, Dempsey, Jensen, de Souza, Karla, Raghunath, Anandita, Rivet, Vernon, DeMoes, Noah, Alejos, Andrés, Wollaber, Allan, 2023. Extensible machine learning for encrypted network traffic application labeling via uncertainty quantification. IEEE Trans. Artif. Intell. 5 (1), 420–433.

Koumar, Josef, Hynek, Karel, Pešek, Jaroslav, Čejka, Tomáš, 2024. Nettisa: Extended ip flow with time-series features for universal bandwidth-constrained high-speed network traffic classification. Comput. Netw. 240, 110147.

Lashkari, Arash Habibi, Kaur, Gurdip, Rahali, Abir, 2020. Didarknet: A contemporary approach to detect and characterize the darknet traffic using deep image learning. In: Proceedings of the 2020 10th International Conference on Communication and Network Security. pp. 1–13.

Li, Zhetao, Xiao, Fu, Wang, Shiguo, Pei, Tingrui, Li, Jie, 2018. Achievable rate maximization for cognitive hybrid satellite-terrestrial networks with af-relays. IEEE J. Sel. Areas Commun. 36 (2), 304–313.

Ma, Ningning, Zhang, Xiangyu, Zheng, Hai-Tao, Sun, Jian, 2018. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In: Proceedings of the European Conference on Computer Vision. ECCV, pp. 116–131.

Maalavika, S., Thangavel, Gnanaprakasam, Basheer, Shajahan, 2023. A review on garlic routing and artificial intelligence applications in public network. In: 2023 International Conference on Computer Science and Emerging Technologies. CSET, IEEE, pp. 1–6.

Sandler, Mark, Howard, Andrew, Zhu, Menglong, Zhmoginov, Andrey, Chen, Liang-Chieh, 2018. Mobilenetv2: Inverted residuals and linear bottlenecks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 4510–4520.

Tan, Mingxing, Le, Quoc, 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In: International Conference on Machine Learning. PMLR, pp. 6105–6114.

Tang, Jiaxi, Shivanna, Rakesh, Zhao, Zhe, Lin, Dong, Singh, Anima, Chi, Ed H., Jain, Sagar, 2020. Understanding and improving knowledge distillation. arXiv preprint arXiv:2002.03532.

Valenti, Silvio, Rossi, Dario, Dainotti, Alberto, Pescapè, Antonio, Finamore, Alessandro, Mellia, Marco, 2013. Reviewing traffic classification. In: Data Traffic Monitoring and Analysis: From Measurement, Classification, and Anomaly Detection to Quality of Experience. pp. 123–147.

Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Łukasz, Polosukhin, Illia, 2017. Attention is all you need. Adv. Neural Inf. Process. Syst. 30.

Velan, Petr, Čermák, Milan, Čeleda, Pavel, Drašar, Martin, 2015. A survey of methods for encrypted traffic classification and analysis. Int. J. Netw. Manage. 25 (5), 355–374.

Wang, Kaiyue, Gao, Jian, Lei, Xinyan, 2023. Mtc: A multi-task model for encrypted network traffic classification based on transformer and 1d-cnn. Intell. Autom. Soft Comput. 37 (1).

Wang, Tongze, Xie, Xiaohui, Wang, Wenduo, Wang, Chuyi, Zhao, Youjian, Cui, Yong, 2024. Netmamba: Efficient network traffic classification via pre-training unidirectional mamba. arXiv preprint arXiv:2405.11449.

Wang, Xiaohui, Xiong, Ying, Qian, Xian, Wei, Yang, Li, Lei, Wang, Mingxuan, 2021. Lightseq2: Accelerated training for transformer-based models on gpus. arXiv preprint arXiv:2110.05722.

Wu, Hua, Guo, Shuyi, Cheng, Guang, Hu, Xiaoyan, 2021. Detecting tor bridge from sampled traffic in backbone networks.

Yang, Tien-Ju, Chen, Yu-Hsin, Sze, Vivienne, 2017. Designing energy-efficient convolutional neural networks using energy-aware pruning. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 5687–5695.

Yang, Le, Jiang, Haojun, Cai, Ruojin, Wang, Yulin, Song, Shiji, Huang, Gao, Tian, Qi, 2021. Condensenet v2: Sparse feature reactivation for deep networks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 3569–3578.

Yang, Tong, Jiang, Jie, Liu, Peng, Huang, Qun, Gong, Junzhi, Zhou, Yang, Miao, Rui, Li, Xiaoming, Uhlig, Steve, 2018. Elastic sketch: Adaptive and fast network-wide measurements. In: Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication. pp. 561–575.

Zhang, Xiangyu, Zhou, Xinyu, Lin, Mengxiao, Sun, Jian, 2018. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 6848–6856.